

Bases de Dados – Prof. Eduardo Corrêa

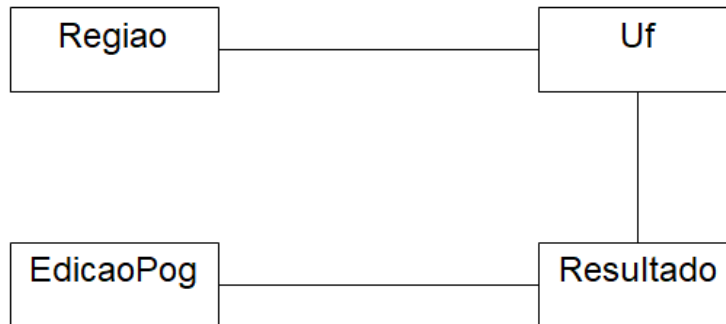
Aula 14 – Subconsultas em Instruções SELECT

1. Descrição da base de dados utilizada na aula

- **Produção de Ovos de Galinha (POG)** é uma pesquisa do IBGE com **edição trimestral** que investiga um painel de informantes composto por todos os estabelecimentos que produzem ovos de galinha e possuem 10.000 ou mais galinhas poedeiras. A produção de ovos compreende tanto ovos para consumo como ovos para incubação. Os dados coletados pela pesquisa são divulgados no SIDRA (sistema de divulgação do IBGE), tabulados por trimestre e Unidade da Federação.
- Nesta aula, você utilizará **subconsultas** da SQL para explorar e analisar uma base de dados que foi montada a partir dos dados reais da POG. A base de dados está disponibilizada no repositório da disciplina com o nome 'pog.db'. Você pode fazer o download e configurá-la no Letos. Os dados englobam 32 edições da pesquisa (de 2010-1 a 2017-4) e estão estruturados em 4 tabelas:

Nome da Relação	Descrição
<i>Regiao</i> (<u>id</u> , nome)	Armazena as regiões do Brasil: {(1,'Norte'), (2, 'Nordeste'), (3, 'Sudeste'), (4, 'Sul'), (5, 'Centro-Oeste')}.
<i>Uf</i> (<u>sigla</u> , nome, idRegiao)	Armazena a sigla, nome e id da região de todas as Unidades da Federação: {'AC','Acre', 1), ('AL', 'Alagoas', 2), ... }.
<i>EdicaoPog</i> (<u>id</u> , ano, trim)	Armazena o id de cada edição em conjunto com seu ano/trimestre de referência: {(1, 2010, 1), (2, 2010, 2), ..., (32, 2017, 4)}
<i>Resultado</i> (<u>idEdicao</u> , <u>siglaUf</u> ,, qtdEstab, qtdPoedeiras, qtdOvos)	Esta é a tabela principal, que contém os resultados divulgados pela pesquisa. Cada resultado da POG refere-se a uma UF (sgl_uf) e edição (id_edicao), englobando até três variáveis: • qtdEstab: quantidade de estabelecimentos com mais de 10.000 galinhas poedeiras. • qtdPoedeiras: número total de galinhas poedeiras. • qtdOvos: quantidade produzida de ovos de galinhas (em mil dúzias)

- O diagrama a seguir indica as associações entre as tabelas.



2. Subconsultas

Uma subconsulta (*subquery*) consiste em um comando SELECT que existe dentro de um outro comando SQL. O comando externo é chamado de comando “pai” da subconsulta. Esta aula apresenta exemplos práticos de utilização de diferentes tipos de subconsulta com a utilização da base de dados da POG, descrita a seguir.

O uso mais comum para as subconsultas é dentro de outra instrução SELECT, com o objetivo de retornar resultados que, de alguma forma, serão utilizados pela consulta pai. No entanto, as *subqueries* também são aceitas em comandos INSERT, UPDATE, DELETE e no CREATE TABLE. Esta aula aborda apenas o uso de subconsultas no SELECT, o chamado “SELECT dentro de SELECT”. Nesta forma de utilização, as subconsultas podem ser de diferentes tipos:

- **Subconsulta de única linha** (*single-row*): retornará sempre uma única linha em seu resultado (independente do número de colunas).
- **Subconsulta de múltiplas linhas** (*multiple-row*): pode retornar zero, uma ou mais linhas em seu resultado (independente do número de colunas). Não é garantido que muitas linhas serão retornadas, mas como isto pode acontecer, a consulta pai deverá estar estruturada para receber muitas linhas.
- **Subconsulta de única coluna** (*single-column*): retornará sempre uma única coluna no resultado (independente do número de linhas).
- **Subconsulta de múltiplas colunas** (*multiple-column*): pode retornar uma ou mais colunas em seu resultado (independente do número de linhas).

- **Subconsulta escalar:** retornará sempre um resultado contendo uma linha e uma única coluna (ou seja, um único valor). Desta forma, uma subconsulta escalar corresponde a uma subconsulta *single-row, single-column*.
- **Subconsulta correlacionada** (*correlated subquery*): este tipo de subconsulta contém uma referência para uma ou mais colunas da tabela pai dentro de sua própria definição. Normalmente, uma condição especificada no WHERE da subconsulta envolverá a comparação de uma coluna da consulta pai com uma coluna da subconsulta, permitindo assim, a efetivação de associações entre uma linha da tabela pai com uma linha gerada pela subconsulta.

3. Subconsultas escalares na Cláusula WHERE

Suponha que você deseja obter uma relação de todas as UF's da primeira edição da POG cuja produção de ovos foi superior à média de todas as UFs. Neste caso você poderia pensar a princípio que seria necessário utilizar dois SELECTs para resolver o problema, conforme mostra o exemplo abaixo:

Exemplo 1a: UFs da 1ª edição com produção acima da média - solução com 2 SELECT's

```
--Select 1
SELECT AVG(qtdOvos) FROM Resultado
WHERE idEdicao=1;

AVG(qtdOvos)
27390.86

--Select 2
SELECT siglaUf, qtdOvos FROM Resultado
WHERE idEdicao=1 AND qtdOvos > 27390.86;
```

siglaUF	qtdOvos
ES	40192
GO	33802
MG	73348
PR	56338
RS	46393
SC	30437
SP	181456

O primeiro SELECT serviu para descobrirmos o valor médio da produção de ovos, que é 27.390,86, um valor escalar. Em seguida, elaboramos um segundo SELECT onde esse valor escalar foi utilizado na cláusula WHERE, mais especificamente na condição `qtdOvos > 27390.86`.

Entretanto, a SQL nos permite combinar esses dois SELECTs em um único comando, bastando para isso transformar o primeiro SELECT em uma subconsulta que retornará um valor escalar para o primeiro SELECT:

Exemplo 1b: UFs da 1ª edição com produção acima da média - solução com subconsulta

```
SELECT siglaUf, qtdOvos FROM Resultado
WHERE idEdicao=1
AND qtdOvos > (SELECT AVG(qtdOvos) FROM Resultado
               WHERE idEdicao=1);
```

siglaUF	qtdOvos
ES	40192
GO	33802
MG	73348
PR	56338
RS	46393
SC	30437
SP	181456

A **consulta interna (subconsulta)** é executada primeiro pelo SGBD (por isso mesmo, ela é está especificada entre parênteses). O resultado de sua execução corresponde a uma relação contendo uma linha e uma coluna (valor escalar), que armazena o valor médio. Tendo sido determinado, este valor médio é então utilizado no WHERE da consulta externa.

Veja que o valor retornado pela subconsulta foi utilizado em um teste do tipo “maior que” (operador “>”). Este tipo de teste pode ser utilizado apenas para avaliar um **valor escalar**. Caso a subquery não fosse do tipo escalar (retornasse mais de uma linha ou mais de uma coluna), o teste com “>” ficaria sem sentido e ocorreria um erro de execução do comando SQL! Apenas as subqueries do tipo escalar podem ser utilizadas em testes que envolvam os operadores =, <>, <, >, >= e <=. O Exemplo 2, apresenta uma subconsulta que identifica a edição/UF com a maior produção de ovos registrada. Como o teste da consulta externa é feito com “=”, a subconsulta precisa ser escalar):

Exemplo 2: Dados da edição/UF que registrou a maior produção de ovos dentre todo o histórico de resultados da POG.

```
SELECT * FROM Resultado
WHERE qtdOvos = (SELECT MAX(qtdOvos) FROM Resultado);
```

idEdicao	siglaUf	qtdEstab	qtdPoedeiras	qtdOvos
32	SP	377	44907216	254014

As subconsultas podem ser utilizadas normalmente em qualquer tipo de SELECT, incluindo os que possuem JOIN, GROUP BY, etc., como mostra o Exemplo 3.

Exemplo 3: retorna o ano, trimestre, número da edição, sigla da UF e quantidade de ovos da edição/UF que registrou a maior produção de ovos dentre todo o histórico da POG

```
SELECT id, ano, trim, siglaUf, qtdOvos
FROM Resultado r INNER JOIN EdicaoPog e ON (e.id = r.idEdicao)
WHERE qtdOvos = (SELECT MAX(qtdOvos) FROM Resultado);
```

id	ano	trim	siglaUF	qtdOvos
32	2017	4	SP	254014

Podemos elaborar instruções contendo mais de uma subconsulta, como mostra o Exemplo 4. Para a maioria dos SGBDs, não há um limite máximo de subconsultas por comando, mas sim um limite máximo de caracteres que podem ser utilizados para definir um comando SQL. Como esse limite costuma ser bem alto, podendo ultrapassar a casa dos milhões de caracteres, é possível dizer que podemos criar um SQL contendo “infinitas” subconsultas.

Exemplo 4: nomes das UF's da 1ª edição com produção acima da média e quantidade de estabelecimentos abaixo da média

```
SELECT nome, qtdOvos, qtdEstab
FROM Resultado r INNER JOIN Uf u ON (u.sigla = r.siglaUf)
WHERE idEdicao=1
AND qtdOvos > (SELECT AVG(qtdOvos) FROM Resultado
               WHERE idEdicao=1)
AND qtdEstab < (SELECT AVG(qtdEstab) FROM Resultado
               WHERE idEdicao = 1);
```

nome	qtdOvos	qtdEstab
Goiás	33802	30

4. Subconsultas *multi-row* na Cláusula WHERE

Uma subconsulta *multi-row* é aquela que retorna mais de uma linha. O operador IN é um dos poucos operadores que podem ser utilizados com este tipo de subconsulta na cláusula WHERE, pois ele é aplicado sobre listas de valores. No exemplo a seguir, produzimos um relatório com todos os resultados das UF's do Rio de Janeiro em todas as edições referentes ao 4º trimestre entre os anos de 2013 e 2017. Nesse caso, utilizamos uma subconsulta que retorna 5 linhas.

Exemplo 5: retorna os resultados do Rio de Janeiro, no 4º trimestre dos anos de 2013 a 2017

```
SELECT * FROM Resultado
WHERE idEdicao IN (SELECT id FROM EdicaoPog
                  WHERE ano BETWEEN 2013 AND 2017
                  AND trim=4)
AND siglaUf = 'RJ';
```

idEdicao	siglaUf	qtdEstab	qtdPoedeiras	qtdOvos
16	RJ	5	390027	1667
20	RJ	6	410385	1613
24	RJ	6	466219	2138
28	RJ	6	262116	1573
32	RJ	6	227925	1308

Além do IN, também é possível utilizar os operadores ANY, SOME e ALL para lidar com subconsultas *multi-row* no WHERE. Porém esses operadores não são tão utilizados na prática e não são suportados pelo SQLite (maiores informações em: https://www.w3schools.com/sql/sql_any_all.asp)

5. Subconsultas *multi-column* na Cláusula WHERE

As subconsultas apresentadas até agora, retornavam apenas uma coluna. Mas a SQL permite com que você monte um teste no WHERE comparando várias colunas de uma vez. Um exemplo envolvendo uma subquery do tipo *multi-row, multi-column* é apresentado abaixo. Observe que a consulta externa utiliza um par de colunas entre parênteses (sgl_uf, id_edicao) que são utilizados na comparação com cada par de resultados retornados pela subconsulta.

Exemplo 6: retorna todos os resultados do Rio de Janeiro e Espírito Santo nas três primeiras edições.

```
SELECT * FROM Resultado
WHERE (siglaUf, idEdicao) IN (SELECT siglaUf, idEdicao
                             FROM Resultado
                             WHERE siglaUf IN ('RJ','ES')
                             AND idEdicao < 4);
```

idEdicao	sgl_uf	qtdEstab	qtdPoedeiras	qtdOvos
1	ES	91	6878665	40192
2	ES	90	7173654	41275
3	ES	88	7700930	43081
1	RJ	7	451703	1650
2	RJ	7	422237	1555
3	RJ	6	338707	1607

Obs.: a consulta proposta não precisaria ser resolvida com o uso de subconsulta, já que existem formas mais simples! Fizemos assim apenas para apresentar a sintaxe de um SQL contendo uma subconsulta multi-column.

6. Subconsultas na Cláusula FROM

Subconsultas podem ser utilizadas na cláusula FROM, com o intuito de gerar resultados que poderão ser utilizados como se fossem uma tabela real (por este motivo uma subconsulta é normalmente chamada de “tabela virtual”). Esta forma de utilização de subconsultas é muito comum.

No exemplo a seguir, uma subconsulta é utilizada no FROM para permitir o cálculo do percentual da produção de ovos de cada UF em relação ao total Brasil de cada edição.

Exemplo 7: Para cada edição/UF, retorna a produção de ovos e percentual da produção da UF em relação ao total Brasil da edição.

```
SELECT r.idEdicao, siglaUf, qtdOvos, tot_brasil,
       ((qtdOvos * 100.0) / tot_brasil) AS pct_prod
FROM resultado r
INNER JOIN (SELECT idEdicao, SUM(qtdOvos) AS tot_brasil
            FROM Resultado GROUP BY idEdicao) t
ON (R.idEdicao = t.idEdicao)
ORDER BY 1, 2;
```

	id_edicao	siglaUf	qtdOvos	tot_brasil	pct_prod
1	1	AC	NULL	602599	NULL
2	1	AL	6093	602599	1.01
3	1	AM	16382	602599	2.72
4	1	BA	8012	602599	1.33
5	1	CE	25074	602599	4.16
6	1	DF	3920	602599	0.65
7	1	ES	40192	602599	6.67
8	1	GO	33802	602599	5.61
9	1	MG	73348	602599	12.17
10	1	MS	8550	602599	1.42
...	...	...	...	...	...

Como essa consulta é resolvida? Como acontece com todo SQL, o processamento começa no FROM, onde existe um INNER JOIN entre a tabela *Resultado* (apelidada de “r”) com uma subconsulta (apelidada de “t”). Esta subconsulta retorna dois campos: id da edição e a somatório da produção de ovos da edição, que é apelidado de “tot_brasil” dentro da subconsulta. Se você pegar esta subconsulta e executá-la isoladamente, verá que ela produz o resultado abaixo:

```
SELECT idEdicao, SUM(qtdOvos) AS tot_brasil
FROM Resultado GROUP BY idEdicao
```

	id_edicao	tot_brasil
1	1	602599
2	2	609524
3	3	620974
4	4	620504
5	5	618977
6	6	632906
7	7	649747
8	8	660772
9	9	673164
10	10	673353
11	11	682835
12	12	666127
13	13	670500
14	14	680807
15	15	692741
16	16	696274
17	17	687596
18	18	698369
19	19	720095
20	20	718818
21	21	706408
22	22	721462
23	23	751423
24	24	748049
25	25	754754
26	26	758993
27	27	776089
28	28	796080
29	29	785053
30	30	814444
31	31	837186
32	32	848826

Em nosso exemplo, esse conjunto de resultados é gerado no FROM, apelidado de “t” e, assim, utilizado normalmente em um JOIN com “resultado”. Embora “t” não seja uma tabela real (mas sim, virtual), não há qualquer problema em utilizar seu conjunto de resultados em uma operação de junção.

7. Subconsulta Correlacionada

Todos os exemplos apresentados até agora envolveram o uso de subconsultas autônomas, isto é subconsultas que executariam de forma bem-sucedida caso fosse separadas da consulta pai.

De maneira oposta, as subconsultas correlacionadas incluem referências para elementos da consulta pai e, por isso, não podem ser executadas de forma independente. Suponha, por exemplo, que você deseja listar, para cada edição da POG, todas as UFs cuja produção de ovos tenha sido superior à produção média de sua edição. Existem algumas formas diferentes para resolver esse problema e uma das mais “naturais” é exatamente com o uso de uma subconsulta correlacionada, como mostra o Exemplo 8:

Exemplo 8: Utilizando uma subconsulta correlacionada para produzir um relatório que mostra as UFs com produção de ovos superior à média de sua edição.

```
SELECT idEdicao, siglaUF, qtdOvos
FROM Resultado a
WHERE a.qtdOvos > (SELECT AVG(qtdOvos)
                   FROM Resultado
                   WHERE idEdicao = a.idEdicao)
ORDER BY 1, 2;
```

	idEdicao	siglaUF	qtdOvos
1	1	ES	40192
2	1	GO	33802
3	1	MG	73348
4	1	PR	56338
5	1	RS	46393
6	1	SC	30437
7	1	SP	181456
8	2	ES	41275
9	2	GO	33227
10	2	MG	72939
11	2	PE	27947
12	2	PR	56632
13	2	RS	48840
14	2	SC	32507
15	2	SP	186562
16	3	ES	43081
17	3	GO	32758
18	3	MG	74775
19	3	PR	58170
20	3	RS	49280
21	3	SC	30352
22	3	SP	193772
23	4	ES	41011
24	4	GO	34029
25	4	MG	737951
...	...	...	...

Como essa consulta funciona? Primeiro observe com atenção a *subquery*:

```
SELECT AVG(qtdOvos)
FROM Resultado
WHERE idEdicao = a.idEdicao
```

Veja o WHERE... Você percebeu o uso do apelido “a.” no lado direito? Isto corresponde a uma referência para uma coluna da consulta pai (e não da *subquery*). Se você observar a consulta pai, verá que ela realmente foi apelidada de “a”. Isto é o que chamamos de “correlação” da subconsulta correlacionada! Esta subconsulta **não** pode ser executada isoladamente, uma única vez, retornando um conjunto de resultados para a consulta externa (como ocorreu em todos os exemplos das Seções 2 a 5). Ao invés disso, **a subconsulta correlacionada é sempre reexecutada para cada linha gerada pela consulta pai** (melhor

explicando: se a consulta pai retornar n linhas, a subconsulta será executada n vezes, uma para cada linha).

Em nosso exemplo, a cada iteração (cada linha da consulta pai), a subconsulta receberá o valor do “idEdicao” da consulta pai e utilizará esse valor no teste de seu WHERE (WHERE idEdicao = a.idEdicao). Com isto, será computada a média da produção de ovos referente à edição da **linha que está sendo correntemente processada na consulta pai**. Uma vez computado, este resultado poderá ser normalmente utilizado no teste da consulta externa (WHERE a.qtdOvos > ...) permitindo com que seja possível identificar se a produção de ovos da linha correntemente processada é maior do que a média de produção de ovos de sua edição correspondente. Veja que a consulta externa e a subconsulta correlacionada “conversam” e trabalham de forma colaborativa para obter os resultados.

8. Operadores EXISTS e NOT EXISTS

O operador EXISTS testa a existência de uma linha em uma subconsulta correlacionada. Se nenhuma linha é encontrada, o resultado é FALSE. Caso contrário, TRUE. O NOT EXISTS reverte os resultados.

Exemplo 9: Utilizando um o operador NOT EXISTS para identificar as UFs que não participaram de nenhuma edição da POG.

```
SELECT nome, sigla
FROM uf u
WHERE NOT EXISTS (SELECT 1 FROM resultado WHERE siglaUF = u.sigla);

    nome sigla
1 Amapá      AP
```

A subconsulta pode retornar qualquer coisa no SELECT (no exemplo acima, estamos simplesmente fazendo SELECT 1, mas poderia ser SELECT *, SELECT ‘x’, etc.). Isso não importa para o EXISTS / NOT EXISTS! Para fazer o contrário, ou seja, listar todas as UFs que participaram de ao menos uma edição da POG, basta trocar de NOT EXISTS para EXISTS.

```
SELECT nome, sigla
FROM uf u
WHERE EXISTS (SELECT 1 FROM Resultado WHERE siglaUf = u.sigla);

    nome sigla
1      Acre  AC
2    Alagoas AL
3   Amazonas AM
4      Bahia BA
5     Ceará  CE
6 Distrito Federal DF
7  Espírito Santo ES
8      Goiás  GO
```

9	Maranhão	MA
10	Minas Gerais	MG
11	Mato Grosso do Sul	MS
12	Mato Grosso	MT
13	Pará	PA
14	Paraíba	PB
15	Pernambuco	PE
16	Piauí	PI
17	Paraná	PR
18	Rio de Janeiro	RJ
19	Rio Grande do Norte	RN
20	Rondônia	RO
21	Roraima	RR
22	Rio Grande do Sul	RS
23	Santa Catarina	SC
24	Sergipe	SE
25	São Paulo	SP
26	Tocantins	TO

Este tipo de consulta é conhecido como *semijoin* – um comando SELECT que utiliza o EXISTS ou NOT EXISTS para comparar linhas de uma tabela com linhas de outra tabela.